LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Bakil Amru(2509106044)

Kelas (A`25)

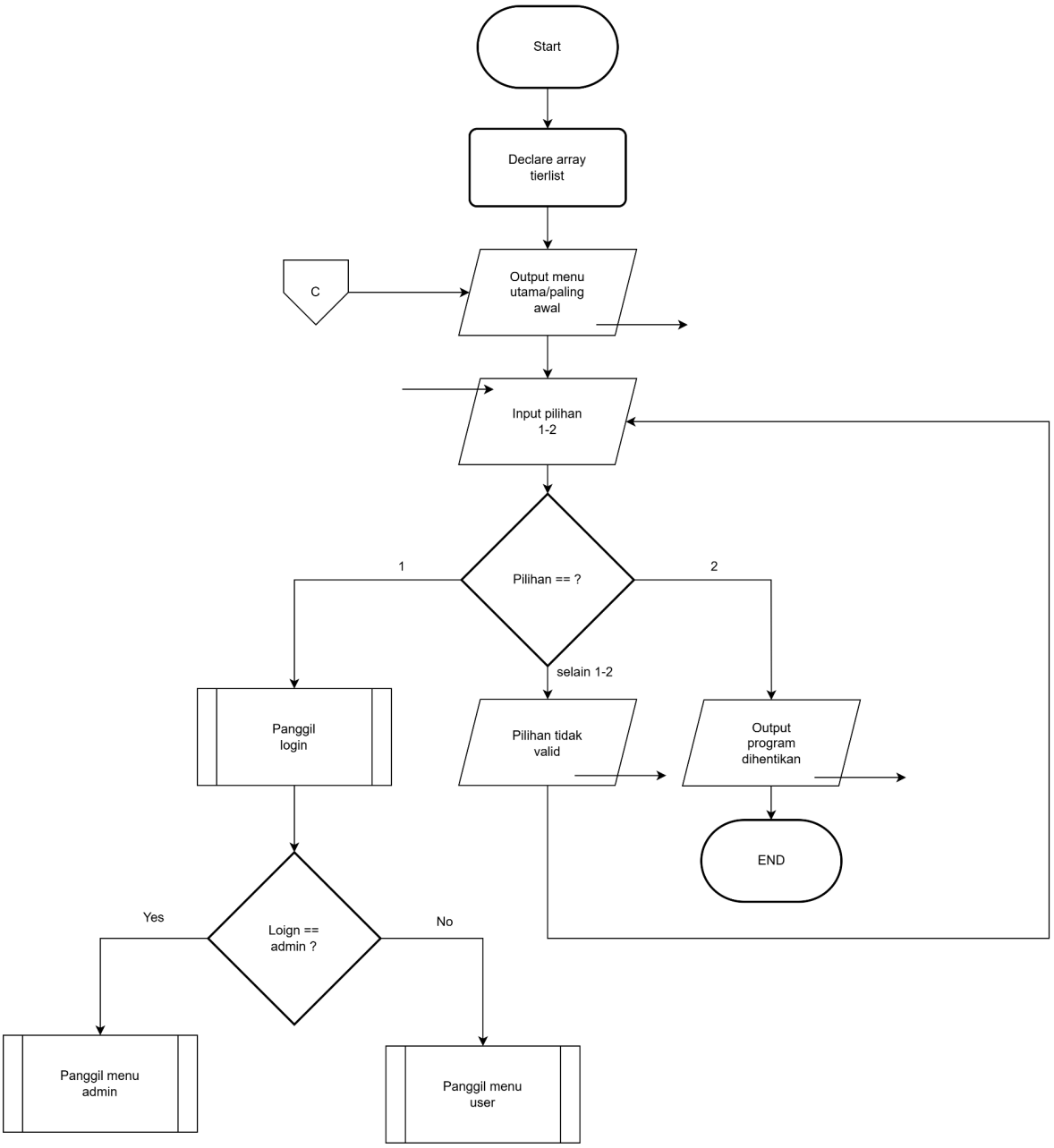
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart 1 (menu utama)



Penjelasan:

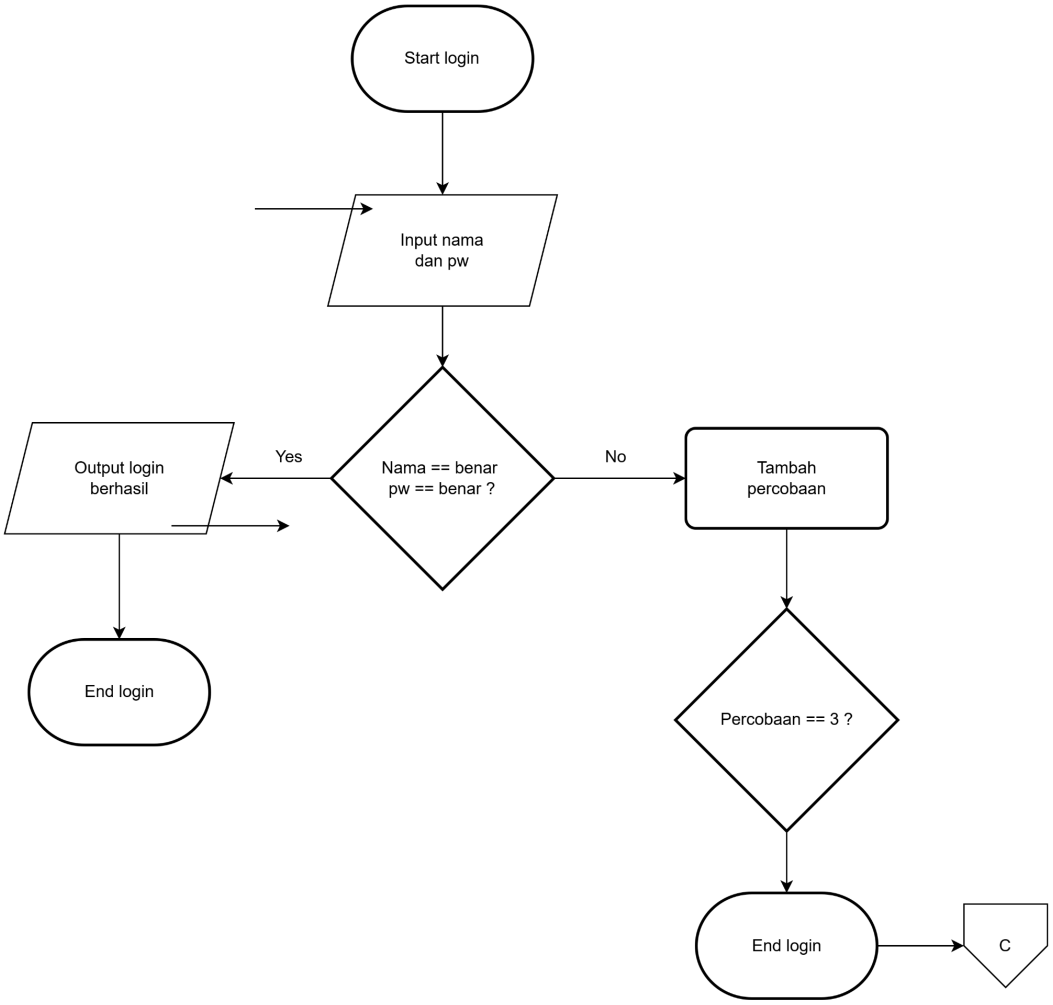
Sama seperti Posttest saya sebelumnya pada bagian ini kita dapat menampilkan menu utama atau menu paling awal jika kita menjalankan program yaitu menu masuk pada program atau keluar,dilanjut dengan diminta untuk menginputkan pilihan kita 1 atau 2 untuk melanjutkan program

jika kita memilih 1 program akan memanggil fungsi login, untuk menentukan kita akan login sebagai admin atau user (detailnya di flowchart kedua), selanjutnya jika kita login sebagai admin program akan memanggil fungsi admin dengan menampilkan menu admin (flowchart ketiga) dan jika kita login sebagai user program pun akan memanggil fungsi user dengan menampilkan menu user (flowchart keempat).

jika kita memilih opsi 2 yaitu keluar dari program maka program akan lansung terhenti, selain pilihan kedua itu akan menghasilkan output tidak valid

catatan penting disitu ada offpage C yang dimana itu nanti akan ada juga pada bagian flowchart lain karena itu adalah pilihan jika kita memilih keluar pada program/menu admin ataupun user, jika kita memilih keluar pada menu admin ataupun user program akan loop ke meenu yang paling awal itu untuk memastikan apakah anda ingin benar benar keluar dari program atau tidak.

2. Flowchart 2 (Fungsi login)

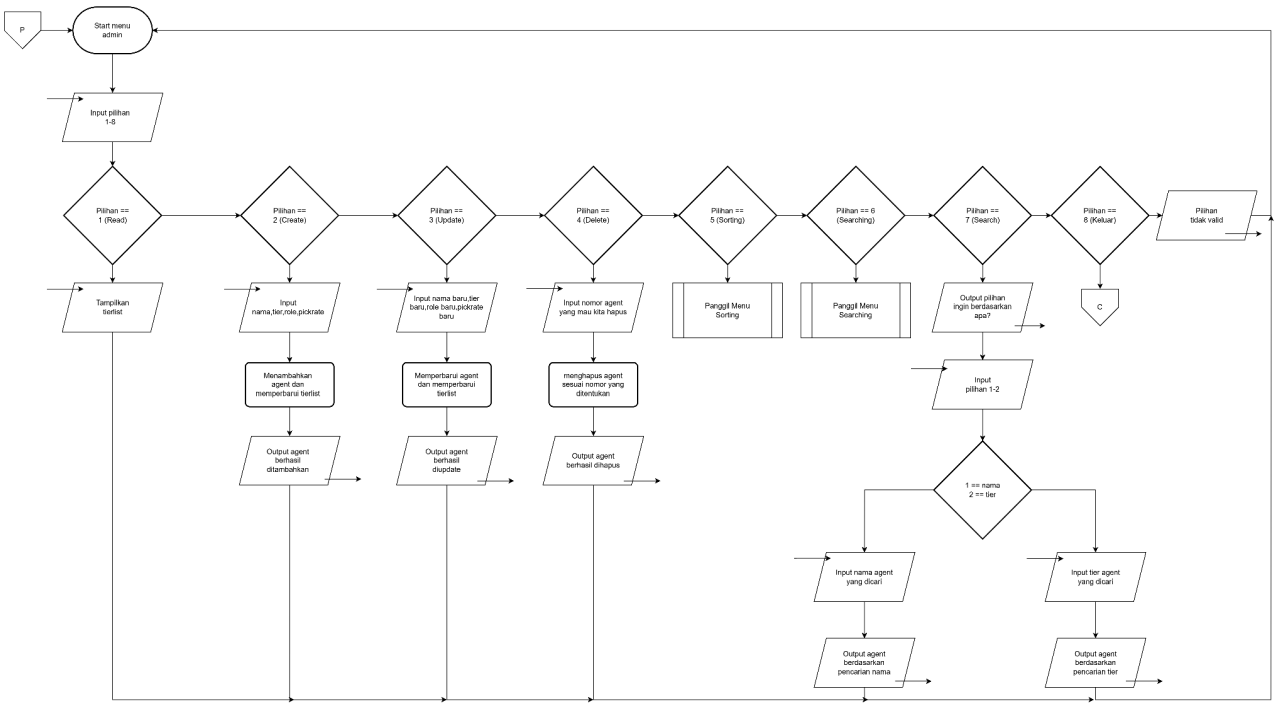


Penjelasan :

Lanjut ke flowchart ke 2 yaitu flowchart fungsi login,diwali dengan diminta menginputkan username dan password, yang dimana jika username dan password sesuai dengan admin, maka menghasilkan output login berhasil dan login sebagai admin, hal yang sama juga berlaku pada username dan password sesuai dengan user maka akan menghasilkan output login berhasil, dan akan lanjut ke menu user.

Apabila salah atau gagal menginputkan akan looping dengan max 3 kali percobaan salah, jika kita sudah salah menginputkan username maupun password/ tidak sesuai, maka program akan menghentikan fungsi login ini dan kita akan looping ke menu paling awal untuk masuk kembali.

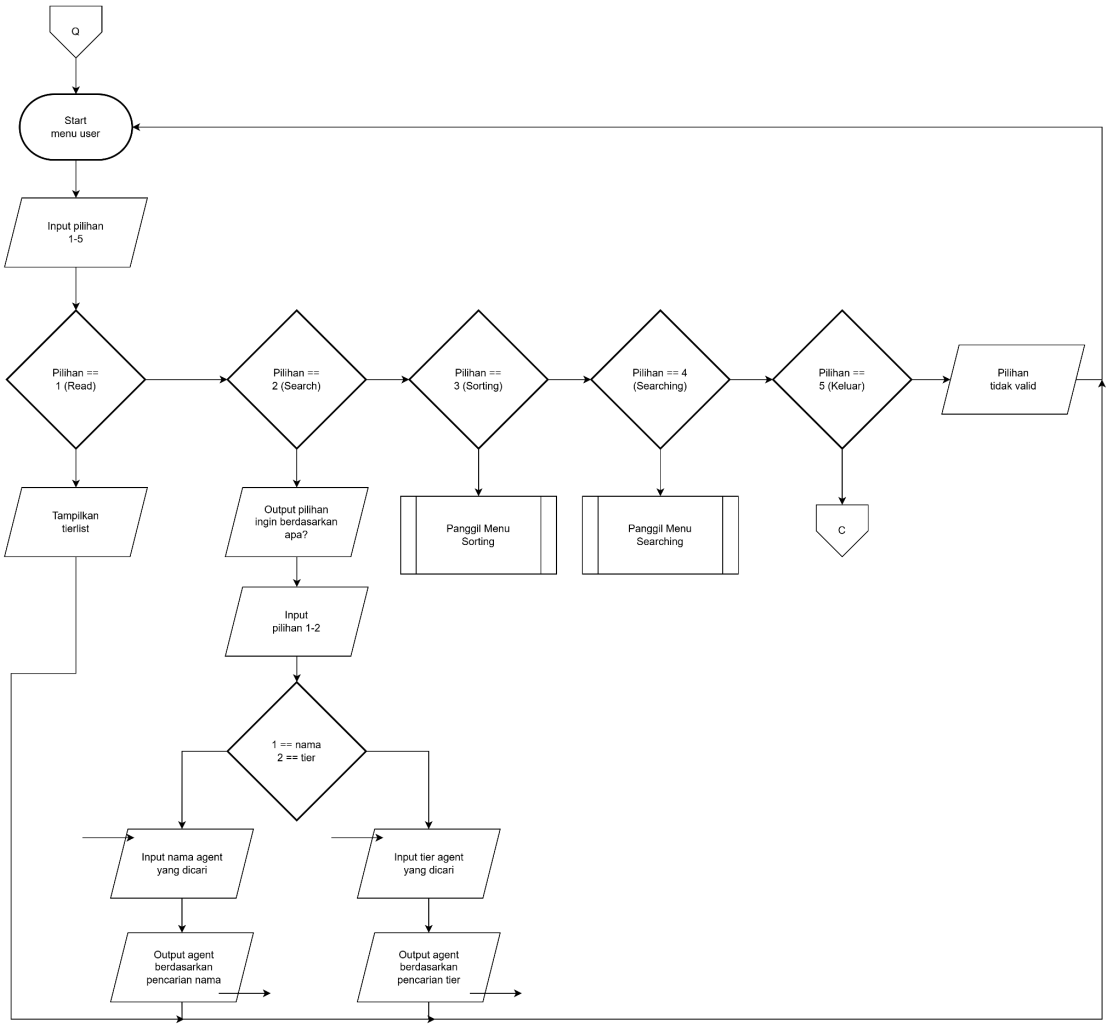
3. Flowchart 3 (Fungsi menu admin)



Penjelasan :

Lanjut untuk fungsi menu admin kita terdapat program CRUD dan penambahan pilihan baru yaitu searching untuk mencari agent mencari berdasarkan pickrate dan role yang mau kita cari. sebelum ke search, kita harusnya sudah tau daari program CRUD sebelumnya jika untuk create yaitu untuk menambahkan agent ke dalam tierlist, read untuk melihat tierlistnya, update untuk mengubah agent yang mau kita ubah/ganti menjadi agen baru dan agent lainnya,dan ada delete untuk menghapus agent yang sudah jarang di pick atau tidak ada lagi di gamenya dsb. disini juga kita bisa melakukan sorting pada agent yang ada di data mau dari tier,pickrate, maupun dari namanya. nah sekarang kita ada searching untuk mencari agent pada tierlist berdasarkan pickrate maupun role nya dan untuk pickrate akan dicari menggunakan binary search dan untuk role akan dicari menggunakan linear search.

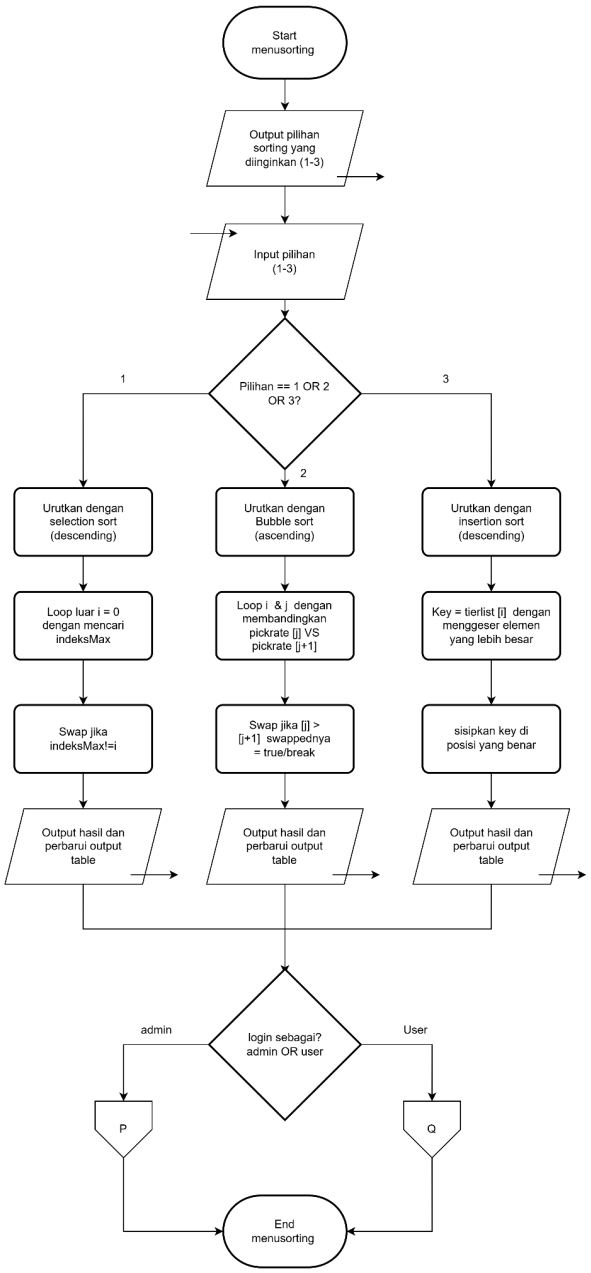
4. Flowchart 4 (Fungsi menu user)



Penjelasan :

lanjut menu user kita terdapat 5 pilihan yaitu read sama seperti pada menu admin yaitu kita dapat melihat tierlist agent agentnya,dan adapula search sama juga seperti pada menu admin yaitu kita dapat memilih mencari berdasarkan apa, dan ketiga ada sorting dengan memanggil menu sortingnya jika 1 atau berdasarkan nama kita menginputkan nama agentnya dan program akan mencari nama agentnya dan menampilkan informasi agentnya, dan jika kita memilih 2 atau cari berdasarkan tier, program akan mencari apa saja agent yang mempunyai tier s dan akan menghasilkan informasi dari agent agent yang mempunyai tier yang kita cari. ada pun pilihan 3 dan 4 itu adalah pilihan untuk memanggil menu sorting (3) dan menu searching (4) pada menu sorting terdapat 3 jenis sorting dan pada searching terdapat 2 jenis searching. dan yang terakhir kita bisa keluar dari menu admin (5)

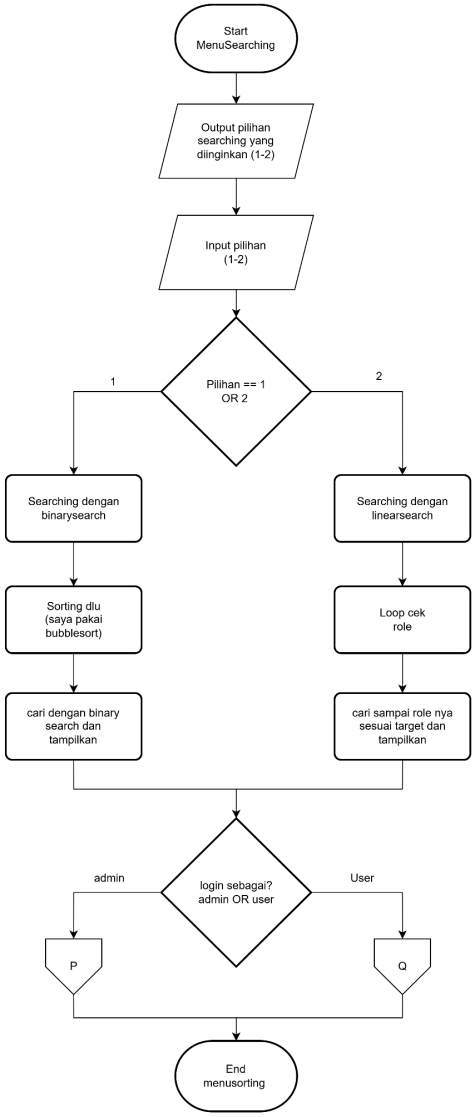
5. Flowchart 5 (Menu Fungsi Sorting)



Penjelasan :

Lalu ada fungsi menu sorting, dia juga berada di menu user dan admin jika dipanggil maka akan menampilkan menu user dan diminta untuk menginput pilihan sorting data kita berdasarkan apa, dengan 3 pilihan yaitu sorting dengan selection sort (descending) untuk mengurutkan sesuai role, bubble sort (ascending) untuk mengurutkan pickrate agent valorantnya,dan insertion sort (descending) untuk mengurutkan sesuai nama agentnya, dan akan dihitung sesuai dengan cara hitung di setiap sorting, setelah di sorting untuk dapat melihat sortingnya kita bisa melihat di menu read jika sudah ter-sorting nantinya. dan terakhir akan kembali sesuai apa yang kita login dahulu (admin atau user) lalu program searching pun akan dihentikan.

6. Flowchart 6 (Menu Fungsi Searching)



Penjelasan :

Lalu kita punya menu searching, walaupun kita juga punya fitur atau emnu search juga tapi kali ini fitur seaarching kita untuk mencari pickrate dan role pada agent yang kita punya menggunakan pointer yang dimana pickrate menggunakan binarySearch dan berdasarkan role akan dicari menggunakan linearSearch, jika sudah dan ketemu akan ditampilkan sesuai dengan pickrate atau role yang kita cari untuk masing masing agent yang ingin kita cari. dan akan kembali looping lagi ke menu admin atau user sesuai apa yang kita login sebelumnya. dan program menuSearching dihentikan.

2. Deskripsi Singkat Program

Program ini hampir sama dengan program sebelumnya namun pada program ini terdapat hal yang baru seperti searching walaupun program sebelumnya juga ada fitur search yang bisa kita liat pada source codenya dan untuk program CRUD ini adalah program CRUD sebelumnya namun sekarang untuk program programnya saya pisah menjadi beberapa subprogram, ada penambahan baru pada menu pilihan CRUD admin maupun user, yaitu fitur mencari agent berdasarkan nama maupun tiernya, yang dimana ini termasuk fungsi overloading.

Jadi, pada fitur mencari agent saya pisahkan menjadi 2 bagian yaitu untuk mencari berdasarkan nama dan tier jika kita mencari sesuai nama, dan program menemukan nama agent yang kita cari maka akan menghasilkan output agent yang kita cari, dan akan menghasilkan informasi lengkap dari agent tersebut dari nama,tier, role ,pickratenya. hal ini juga berlaku pada mencari berdasarkan tier, program juga akan menghasilkan informasi lengkap terkait agent yang kita cari, dengan catatan jika agent nya ada pada tierlist, fungsi - fungsi yang lain kurang lebih sama, namun yang mungkin sedikit berbeda jika pada menu kita memilih keluar, program tidak akan otomatis terhenti namun dia looping ke bagian menu yang paling awal, dan saya juga ada menambahkan sebuah menu yang paling awal dengan pilihan masuk ke program tierlist, dilanjut dengan login. Atau keluar dari program, pilihan keluar pada program ini saja yang akan menghentikan programnya, jadi jika kita memutuskan untuk keluar pada menu admin maupun user, program akan kembali ke menu ini/menu paling awal pada program untuk memastikan jika orang yang sudah login benar benar ingin keluar pada program, artinya jika kita sudah masuk pada program dan kita ingin benar benar menghentikan program kita harus keluar 2 kali, menu admin/user dan menu yang paling awal.

Dan untuk sorting nya saya ada 3 jenis sorting yaitu yaitu Selection sort untuk mengurutkan Tier, dan ada bubble sort untuk mengurutkan persentase pickrate (seberapa banyak di pilih), dan ada insertion sort untuk mengurutkan nama-nama agentnya

Untuk Searching saya tambahkan 2 jenis searching yaitu binarysearch dan linear search untuk mencari pickrate dan role dari sebuah agent yang ingin kita cari

3. Source Code

3.1 Source Code (CPP)

```
#include <iostream>

#include <string>

#include <stdexcept>

#include "errorhandling.h"

using namespace std;


#define MAX_AGENT 100

#define MAX_USER  10


struct Statistik {

    string role;

    int pickrate;

};


struct Agent {

    string nama;

    string tier;

    Statistik stat;

};


struct User {

    string username;

    string password;

    string role;
```

```
};


// ===== SUB PROGRAM =====
void tampilkanTierlistRekursif(Agent tierlist[], int i, int jumlahAgent) {
    if (jumlahAgent == 0) {
        cout << "Belum ada agent yang ditambahkan." << endl;
        return;
    }
    if (i == 0) {
        cout << "\n=== TIERLIST VALORANT ===" << endl;
         cout << "+----+----------------+------+------------+----------+" <<
endl;
         cout << "| No | Nama Agent       | Tier | Role         | Pickrate |"
<< endl;
         cout << "+----+----------------+------+------------+----------+" <<
endl;
    }


    cout << "| " << i+1
         << "  | " << tierlist[i].nama
         << string(15 - tierlist[i].nama.size(), ' ')
         << "| " << tierlist[i].tier
         << "    | " << tierlist[i].stat.role
         << string(11 - tierlist[i].stat.role.size(), ' ')
         << "| " << tierlist[i].stat.pickrate << "%"
         << string(8 - to_string(tierlist[i].stat.pickrate).size(), ' ')
         << "|" << endl;
```

```
    if (i == jumlahAgent - 1) {
         cout << "+----+----------------+------+-------------+----------+" <<
endl;
        return;
    }
    tampilkanTierlistRekursif(tierlist, i+1, jumlahAgent);
}

// ===== TAMBAH AGENT =====
void tambahAgent(Agent tierlist[], int *jumlahAgent) {
    try {
        if (*jumlahAgent >= MAX_AGENT)
            throw KapasitasPenuhException();

        string nama, tier, role;
        int pickrate;

        cout << "Nama Agent: ";
        cin.ignore();
        getline(cin, nama);
        validasiTidakKosong(nama, "Nama");

        cout << "Tier (S/A/B/C/D): ";
        getline(cin, tier);
        validasiTidakKosong(tier, "Tier");
        validasiTier(tier);
```

```
        cout << "Role: ";
        getline(cin, role);
        validasiTidakKosong(role, "Role");

        cout << "Pickrate (%): ";
        if (!(cin >> pickrate)) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error("Input pickrate harus berupa angka.");
        }
        validasiPickrate(pickrate);

        tierlist[*jumlahAgent] = {nama, tier, {role, pickrate}};
        (*jumlahAgent)++;
            cout << ">> Agent \"" << nama << "\" berhasil ditambahkan ke
daftar.\n";

    } catch (KapasitasPenuhException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (InputKosongException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (TierException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (PickrateException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (runtime_error &e) {
```

```
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== PROSES UPDATE DATA =====
void prosesUpdateData(Agent *ptrAgent) {
    try {
        string nama, tier, role;
        int pickrate;

        cout << "Nama baru: ";
        cin.ignore();
        getline(cin, nama);
        validasiTidakKosong(nama, "Nama");

        cout << "Tier baru (S/A/B/C/D): ";
        getline(cin, tier);
        validasiTidakKosong(tier, "Tier");
        validasiTier(tier);

        cout << "Role baru: ";
        getline(cin, role);
        validasiTidakKosong(role, "Role");

        cout << "Pickrate baru (%): ";
        if (!(cin >> pickrate)) {
```

```
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error("Input pickrate harus berupa angka.");
        }
        validasiPickrate(pickrate);

        ptrAgent->nama = nama;
        ptrAgent->tier = tier;
        ptrAgent->stat.role = role;
        ptrAgent->stat.pickrate = pickrate;
        cout << ">> Data agent berhasil diperbarui.\n";

    } catch (InputKosongException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (TierException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (PickrateException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (runtime_error &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== UBAH AGENT =====
void ubahAgent(Agent tierlist[], int jumlahAgent) {
    try {
```

```
        if (jumlahAgent == 0)
            throw runtime_error("Belum ada agent yang bisa diubah.");

        int nomorAgent;
        cout << "Nomor agent yang diubah: ";
        if (!(cin >> nomorAgent)) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error("Input nomor harus berupa angka.");
        }
        validasiNomor(nomorAgent, jumlahAgent);
        prosesUpdateData(&tierlist[nomorAgent - 1]);

    } catch (NomorTidakValidException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (runtime_error &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== HAPUS AGENT =====
void hapusAgent(Agent tierlist[], int *jumlahAgent) {
    try {
        if (*jumlahAgent == 0)
            throw runtime_error("Belum ada agent yang bisa dihapus.");
```

```
        int nomorAgent;
        cout << "Nomor agent yang dihapus: ";
        if (!(cin >> nomorAgent)) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error("Input nomor harus berupa angka.");
        }
        validasiNomor(nomorAgent, *jumlahAgent);

        string namaHapus = tierlist[nomorAgent - 1].nama;
        for (int i = nomorAgent - 1; i < *jumlahAgent - 1; i++) {
            tierlist[i] = tierlist[i + 1];
        }
        (*jumlahAgent)--;
           cout << ">> Agent \"" << namaHapus << "\" berhasil dihapus dari
daftar.\n";

    } catch (NomorTidakValidException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (runtime_error &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== CARI AGENT =====
void cariAgent(string nama, Agent tierlist[], int jumlahAgent) {
    for (int i = 0; i < jumlahAgent; i++) {
```

```
        if (tierlist[i].nama == nama) {
            cout << "\n=== DATA AGENT DITEMUKAN ===" << endl;
            cout << "Nama     : " << tierlist[i].nama << endl;
            cout << "Tier     : " << tierlist[i].tier << endl;
            cout << "Role     : " << tierlist[i].stat.role << endl;
            cout << "Pickrate : " << tierlist[i].stat.pickrate << "%" << endl;
            return;
        }
    }
    cout << ">> Agent \"" << nama << "\" tidak ditemukan di daftar.\n";
}

void cariAgentTier(string tier, Agent tierlist[], int jumlahAgent) {
    try {
        validasiTier(tier);
        bool ditemukan = false;
        for (int i = 0; i < jumlahAgent; i++) {
            if (tierlist[i].tier == tier) {
                cout << "\n=== DATA AGENT DITEMUKAN ===" << endl;
                cout << "Nama     : " << tierlist[i].nama << endl;
                cout << "Tier     : " << tierlist[i].tier << endl;
                cout << "Role     : " << tierlist[i].stat.role << endl;
                  cout << "Pickrate : " << tierlist[i].stat.pickrate << "%" <<
endl;
                ditemukan = true;
            }
        }
```

```
        if (!ditemukan)
            throw AgentTidakDitemukanException("tier " + tier);

    } catch (TierException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (AgentTidakDitemukanException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== SORTING =====
int nilaiTier(string tier) {
    if (tier == "S") return 5;
    if (tier == "A") return 4;
    if (tier == "B") return 3;
    if (tier == "C") return 2;
    if (tier == "D") return 1;
    return 0;
}

void sortTierDescending(Agent tierlist[], int jumlahAgent) {
    for (int i = 0; i < jumlahAgent - 1; i++) {
        int indeksMax = i;
        for (int j = i + 1; j < jumlahAgent; j++) {
                                        if    (nilaiTier(tierlist[j].tier)    >
nilaiTier(tierlist[indeksMax].tier))
                indeksMax = j;
```

```
        }
        if (indeksMax != i) {
            Agent temp = tierlist[i];
            tierlist[i] = tierlist[indeksMax];
            tierlist[indeksMax] = temp;
        }
    }
    cout << ">> Daftar berhasil diurutkan berdasarkan Tier (S -> D).\n";
}

void sortPickrateAscending(Agent tierlist[], int jumlahAgent) {
    bool swapped;
    for (int i = 0; i < jumlahAgent - 1; i++) {
        swapped = false;
        for (int j = 0; j < jumlahAgent - i - 1; j++) {
            if (tierlist[j].stat.pickrate > tierlist[j + 1].stat.pickrate) {
                Agent temp = tierlist[j];
                tierlist[j] = tierlist[j + 1];
                tierlist[j + 1] = temp;
                swapped = true;
            }
        }
        if (swapped == false) break;
    }
     cout << ">> Daftar berhasil diurutkan berdasarkan Pickrate (rendah ->
tinggi).\n";
}
```

```
void sortNamaAscending(Agent tierlist[], int jumlahAgent) {
    for (int i = 1; i < jumlahAgent; i++) {
        Agent key = tierlist[i];
        int j = i - 1;
        while (j >= 0 && tierlist[j].nama > key.nama) {
            tierlist[j + 1] = tierlist[j];
            j = j - 1;
        }
        tierlist[j + 1] = key;
    }
    cout << ">> Daftar berhasil diurutkan berdasarkan Nama (A -> Z).\n";
}

// ===== SEARCHING =====
void    binarySearchPickrate(Agent    *tierlist,    int    jumlahAgent,    int
targetPickrate) {
    try {
        validasiPickrate(targetPickrate);

        bool swapped;
        for (int i = 0; i < jumlahAgent - 1; i++) {
            swapped = false;
            for (int j = 0; j < jumlahAgent - i - 1; j++) {
                        if ((tierlist + j)->stat.pickrate > (tierlist + j +
1)->stat.pickrate) {
                    Agent temp = *(tierlist + j);
```

```
                    *(tierlist + j) = *(tierlist + j + 1);
                    *(tierlist + j + 1) = temp;
                    swapped = true;
                }
            }
            if (swapped == false) break;
        }

        int kiri = 0, kanan = jumlahAgent - 1;
        bool ditemukan = false;

        while (kiri <= kanan) {
            int tengah = (kiri + kanan) / 2;
            if ((tierlist + tengah)->stat.pickrate == targetPickrate) {
                int awal = tengah;
                   while (awal > 0 && (tierlist + awal - 1)->stat.pickrate ==
targetPickrate)
                    awal--;
                        cout << "\n=== HASIL BINARY SEARCH (Pickrate: " <<
targetPickrate << "%) ===" << endl;
                        for (int i = awal; i < jumlahAgent && (tierlist +
i)->stat.pickrate == targetPickrate; i++) {
                    cout << "Nama     : " << (tierlist + i)->nama << endl;
                    cout << "Tier     : " << (tierlist + i)->tier << endl;
                       cout << "Role      : " << (tierlist + i)->stat.role <<
endl;
                     cout << "Pickrate : " << (tierlist + i)->stat.pickrate <<
"%" << endl;
```

```
                    cout << "---" << endl;
                }
                ditemukan = true;
                break;
            } else if ((tierlist + tengah)->stat.pickrate < targetPickrate) {
                kiri = tengah + 1;
            } else {
                kanan = tengah - 1;
            }
        }

        if (!ditemukan)
                            throw  AgentTidakDitemukanException("pickrate  "  +
to_string(targetPickrate) + "%");

    } catch (PickrateException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (AgentTidakDitemukanException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

void linearSearchRole(Agent *tierlist, int jumlahAgent, string targetRole) {
    try {
        validasiTidakKosong(targetRole, "Role");
        bool ditemukan = false;
```

```
        cout << "\n=== HASIL LINEAR SEARCH (Role: " << targetRole << ") ===" << endl;
        for (int i = 0; i < jumlahAgent; i++) {
            if ((tierlist + i)->stat.role == targetRole) {
                cout << "Nama     : " << (tierlist + i)->nama << endl;
                cout << "Tier     : " << (tierlist + i)->tier << endl;
                cout << "Role     : " << (tierlist + i)->stat.role << endl;
                cout << "Pickrate : " << (tierlist + i)->stat.pickrate << "%" << endl;
                cout << "---" << endl;
                ditemukan = true;
            }
        }
        if (!ditemukan)
            throw AgentTidakDitemukanException("role \"" + targetRole + "\"");

    } catch (InputKosongException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    } catch (AgentTidakDitemukanException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== MENU SEARCHING =====
void menuSearching(Agent tierlist[], int jumlahAgent) {
    try {
        if (jumlahAgent == 0)
```

```
        throw runtime_error("Belum ada agent untuk dicari.");


    int pilihan;
    cout << "\n+-----------------------------------+" << endl;
    cout << "|          SEARCHING AGENT           |" << endl;
    cout << "+-----------------------------------+" << endl;
    cout << "| 1. Binary Search (cari Pickrate)   |" << endl;
    cout << "| 2. Linear Search (cari Role)       |" << endl;
    cout << "+-----------------------------------+" << endl;
    cout << "Pilihan: ";
    cin >> pilihan;


    if (pilihan == 1) {
        int targetPickrate;
        cout << "Masukkan pickrate yang dicari (%): ";
        if (!(cin >> targetPickrate)) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw runtime_error("Input pickrate harus berupa angka.");
        }
        binarySearchPickrate(tierlist, jumlahAgent, targetPickrate);
    } else if (pilihan == 2) {
        string targetRole;
        cout << "Masukkan role yang dicari: ";
        cin.ignore();
        getline(cin, targetRole);
```

```
            linearSearchRole(tierlist, jumlahAgent, targetRole);
        } else {
            throw runtime_error("Pilihan tidak valid.");
        }

    } catch (runtime_error &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== MENU SORTING =====
void menuSorting(Agent tierlist[], int jumlahAgent) {
    try {
        if (jumlahAgent == 0)
            throw runtime_error("Belum ada agent untuk diurutkan.");

        int pilihan;
        cout << "\n+-----------------------------+" << endl;
        cout << "|         SORTING AGENT        |" << endl;
        cout << "+-----------------------------+" << endl;
        cout << "| 1. Sort Tier (S -> D)        |" << endl;
        cout << "| 2. Sort Pickrate (kecil->bsr)|" << endl;
        cout << "| 3. Sort Nama (A -> Z)        |" << endl;
        cout << "+-----------------------------+" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;
```

```
        switch (pilihan) {
            case 1: sortTierDescending(tierlist, jumlahAgent);    break;
            case 2: sortPickrateAscending(tierlist, jumlahAgent); break;
            case 3: sortNamaAscending(tierlist, jumlahAgent);     break;
            default: throw runtime_error("Pilihan tidak valid.");
        }

    } catch (runtime_error &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
}

// ===== MENU ADMIN =====
int menuAdmin(Agent tierlist[], int *jumlahAgent) {
    int pilihan;
    do {
        cout << "\n+================ MENU ADMIN ================+" << endl;
        cout << "| 1. Lihat Tierlist                          |" << endl;
        cout << "| 2. Tambah Agent                            |" << endl;
        cout << "| 3. Ubah Agent                              |" << endl;
        cout << "| 4. Hapus Agent                             |" << endl;
        cout << "| 5. Sorting Agent                           |" << endl;
        cout << "| 6. Searching Agent                         |" << endl;
        cout << "| 7. Cari Agent                              |" << endl;
        cout << "| 8. Keluar                                  |" << endl;
```

```
        cout << "+==========================================+" << endl;
        cout << "Pilihan: ";

        try {
            if (!(cin >> pilihan)) {
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error("Input harus berupa angka.");
            }
        } catch (runtime_error &e) {
            cout << ">> Kesalahan: " << e.what() << "\n";
            pilihan = 0;
            continue;
        }

        switch (pilihan) {
        case 1: tampilkanTierlistRekursif(tierlist, 0, *jumlahAgent); break;
        case 2: tambahAgent(tierlist, jumlahAgent); break;
        case 3: ubahAgent(tierlist, *jumlahAgent); break;
        case 4: hapusAgent(tierlist, jumlahAgent); break;
        case 5: menuSorting(tierlist, *jumlahAgent); break;
        case 6: menuSearching(tierlist, *jumlahAgent); break;
        case 7: {
            int subPilihan;
            cout << "\n+---------------------------+" << endl;
            cout << "|        CARI AGENT         |" << endl;
```

```
        cout << "+---------------------------+" << endl;
        cout << "| 1. Cari berdasarkan Nama  |" << endl;
        cout << "| 2. Cari berdasarkan Tier  |" << endl;
        cout << "+---------------------------+" << endl;
        cout << "Pilihan: ";
        cin >> subPilihan;
        if (subPilihan == 1) {
            string nama;
            cout << "Masukkan nama agent: ";
            cin.ignore();
            getline(cin, nama);
            cariAgent(nama, tierlist, *jumlahAgent);
        } else if (subPilihan == 2) {
            string tier;
            cout << "Masukkan tier (S/A/B/C/D): ";
            cin.ignore();
            getline(cin, tier);
            cariAgentTier(tier, tierlist, *jumlahAgent);
        } else {
            cout << ">> Kesalahan: Pilihan tidak valid.\n";
        }
        break;
    }
    case 8: cout << "Keluar dari menu Admin.\n"; break;
    default: cout << ">> Kesalahan: Pilihan tidak valid.\n";
    }
```

```
    } while (pilihan != 8);
    return 0;
}

// ===== MENU USER =====
bool menuUser(Agent tierlist[], int jumlahAgent) {
    int pilihan;
    do {
        cout << "\n+================ MENU USER ================+" << endl;
        cout << "| 1. Lihat Tierlist                          |" << endl;
        cout << "| 2. Cari Agent                              |" << endl;
        cout << "| 3. Sorting Agent                           |" << endl;
        cout << "| 4. Searching Agent                         |" << endl;
        cout << "| 5. Keluar                                  |" << endl;
        cout << "+============================================+" << endl;
        cout << "Pilihan: ";

        try {
            if (!(cin >> pilihan)) {
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error("Input harus berupa angka.");
            }
        } catch (runtime_error &e) {
            cout << ">> Kesalahan: " << e.what() << "\n";
            pilihan = 0;
```

```
            continue;
        }

        switch (pilihan) {
        case 1: tampilkanTierlistRekursif(tierlist, 0, jumlahAgent); break;
        case 2: {
            int subPilihan;
            cout << "\n+---------------------------+" << endl;
            cout << "|        CARI AGENT         |" << endl;
            cout << "+---------------------------+" << endl;
            cout << "| 1. Cari berdasarkan Nama  |" << endl;
            cout << "| 2. Cari berdasarkan Tier  |" << endl;
            cout << "+---------------------------+" << endl;
            cout << "Pilihan: ";
            cin >> subPilihan;
            if (subPilihan == 1) {
                string nama;
                cout << "Masukkan nama agent: ";
                cin.ignore();
                getline(cin, nama);
                cariAgent(nama, tierlist, jumlahAgent);
            } else if (subPilihan == 2) {
                string tier;
                cout << "Masukkan tier (S/A/B/C/D): ";
                cin.ignore();
                getline(cin, tier);
```

```
                cariAgentTier(tier, tierlist, jumlahAgent);
            } else {
                cout << ">> Kesalahan: Pilihan tidak valid.\n";
            }
            break;
        }
        case 3: menuSorting(tierlist, jumlahAgent); break;
        case 4: menuSearching(tierlist, jumlahAgent); break;
        case 5: cout << "Keluar dari menu User.\n"; break;
        default: cout << ">> Kesalahan: Pilihan tidak valid.\n";
        }
    } while (pilihan != 5);
    return true;
}

// ===== LOGIN =====
int login(User users[], int jumlahUser) {
    string username, password;
    int attempt = 0;

    try {
        while (attempt < 3) {
                cout << "\n+================= LOGIN ====================+" <<
endl;
            cout << "| Masukkan username dan password untuk login |" << endl;
            cout << "+===========================================+" << endl;
            cout << "Username: ";
```

```
            cin >> username;
            cout << "Password: ";
            cin >> password;

            for (int i = 0; i < jumlahUser; i++) {
                    if (users[i].username == username && users[i].password ==
password) {
                    cout << ">> Selamat datang, " << username
                         << "! (" << users[i].role << ")\n";
                    return i;
                }
            }

            attempt++;
            cout << ">> Username atau password salah. Sisa percobaan: "
                 << (3 - attempt) << "x\n";
        }
        throw LoginGagalException();

    } catch (LoginGagalException &e) {
        cout << ">> Kesalahan: " << e.what() << "\n";
    }
    return -1;
}

// ===== MAIN =====
int main() {
```

```
    Agent tierlist[MAX_AGENT];
    int jumlahAgent = 0;

    User users[MAX_USER];
    int jumlahUser = 2;

    users[0] = {"Bakil", "044", "Admin"};
    users[1] = {"User",  "123", "User"};

    tierlist[jumlahAgent++] = {"Reyna", "A", {"Duelist",   60}};
    tierlist[jumlahAgent++] = {"Jett",  "S", {"Duelist",   85}};
    tierlist[jumlahAgent++] = {"Iso",   "B", {"Duelist",   40}};
    tierlist[jumlahAgent++] = {"Sage",  "B", {"Sentinel",  67}};
    tierlist[jumlahAgent++] = {"Kay-O", "A", {"Initiator", 45}};

    int pilihanUtama;
    do {
        cout << "\n+=========================================+" << endl;
        cout << "|   Selamat datang di Valorant Tierlist  |" << endl;
        cout << "+=========================================+" << endl;
        cout << "| 1. Masuk                                |" << endl;
        cout << "| 2. Keluar                               |" << endl;
        cout << "+=========================================+" << endl;
        cout << "Pilihan: ";

        try {
```

```
            if (!(cin >> pilihanUtama)) {
                cin.clear();
                cin.ignore(1000, '\n');
                throw runtime_error("Input harus berupa angka.");
            }
            if (pilihanUtama == 1) {
                int idxUser = login(users, jumlahUser);
                if (idxUser != -1) {
                    if (users[idxUser].role == "Admin")
                        menuAdmin(tierlist, &jumlahAgent);
                    else
                        menuUser(tierlist, jumlahAgent);
                }
            } else if (pilihanUtama == 2) {
                 cout << ">> Terima kasih sudah menggunakan Valorant Tierlist.
Sampai jumpa!\n";
            } else {
                    throw runtime_error("Pilihan tidak valid. Masukkan 1 atau
2.");
            }
        } catch (runtime_error &e) {
            cout << ">> Kesalahan: " << e.what() << "\n";
            pilihanUtama = 0;
        }
    } while (pilihanUtama != 2);
    return 0;
}
```

3.2 Source Code (.H)

```
#ifndef ERRORHANDLING_H
#define ERRORHANDLING_H


#include <stdexcept>

#include <string>

using namespace std;


// ===== KELAS EXCEPTION BUATAN SENDIRI =====


class PickrateException : public runtime_error {

public:

    PickrateException(int nilai)

        : runtime_error("Pickrate " + to_string(nilai) +

                        " tidak valid! Masukkan angka antara 0 sampai 100.")
{}

};


class TierException : public runtime_error {

public:

    TierException(string tier)

        : runtime_error("Tier \"" + tier +

                       "\" tidak dikenali! Tier yang tersedia hanya S, A, B,
C, atau D.") {}

};


class InputKosongException : public runtime_error {
```

```
public:
    InputKosongException(string field)
        : runtime_error("Kolom " + field + " tidak boleh dibiarkan kosong!")
{}
};

class KapasitasPenuhException : public runtime_error {
public:
    KapasitasPenuhException()
        : runtime_error("Daftar agent sudah penuh! Maksimal hanya 100 agent.")
{}
};

class AgentTidakDitemukanException : public runtime_error {
public:
    AgentTidakDitemukanException(string keterangan)
          : runtime_error("Tidak ada agent dengan " + keterangan + " yang
cocok.") {}
};

class NomorTidakValidException : public runtime_error {
public:
    NomorTidakValidException(int nomor, int maks)
        : runtime_error("Nomor " + to_string(nomor) +
                        " tidak ada dalam daftar! Pilih antara 1 sampai " +
                        to_string(maks) + ".") {}
};
```

```
class LoginGagalException : public runtime_error {
public:
    LoginGagalException()
        : runtime_error("Anda sudah salah memasukkan password sebanyak 3 kali.
Akses ditolak!") {}
};

// ===== FUNGSI VALIDASI =====

void validasiPickrate(int nilai) {
    if (nilai < 0 || nilai > 100)
        throw PickrateException(nilai);
}

void validasiTier(string tier) {
     if (tier != "S" && tier != "A" && tier != "B" && tier != "C" && tier !=
"D")
        throw TierException(tier);
}

void validasiTidakKosong(string input, string namaKolom) {
    if (input.empty())
        throw InputKosongException(namaKolom);
}

void validasiNomor(int nomor, int maks) {
```

```
    if (nomor < 1 || nomor > maks)

        throw NomorTidakValidException(nomor, maks);

}



#endif
```

4. Hasil Output

Gambar 4.1

```
+=========================================+
|   Selamat datang di Valorant Tierlist   |
+=========================================+
| 1. Masuk                                |
| 2. Keluar                               |
+=========================================+
Pilihan:
```

Gambar 4.1
Tampilan paling awal saat program dijalankan

Gambar 4.2

```
+=========================================+
|   Selamat datang di Valorant Tierlist   |
+=========================================+
| 1. Masuk                                |
| 2. Keluar                               |
+=========================================+
Pilihan: 2
Terima kasih, Program selesai.
```

Gambar 4.2
Jika kita memilih 2 (menghentikan program)

Gambar 4.3

```
+=========================================+
|   Selamat datang di Valorant Tierlist   |
+=========================================+
| 1. Masuk                                |
| 2. Keluar                               |
+=========================================+
Pilihan: 1

+================= LOGIN ====================+
| Masukkan username dan password untuk login |
+=============================================+
Username:
```

Gambar 4.3
Jika kita memilih 1 pada menu paling awal (diminta memasukkan usn dan pw)

Gambar 4.4

```
+=========================================+
|   Selamat datang di Valorant Tierlist   |
+=========================================+
| 1. Masuk                                |
| 2. Keluar                               |
+=========================================+
Pilihan: 1

+================= LOGIN ====================+
| Masukkan username dan password untuk login |
+=============================================+
Username: w
Password: w
Username atau password salah.

+================= LOGIN ====================+
| Masukkan username dan password untuk login |
+=============================================+
Username: w
Password: w
Username atau password salah.

+================= LOGIN ====================+
| Masukkan username dan password untuk login |
+=============================================+
Username: putin
Password: w
Username atau password salah.
Login gagal 3 kali. kembali ke menu utama.

+=========================================+
|   Selamat datang di Valorant Tierlist   |
+=========================================+
| 1. Masuk                                |
| 2. Keluar                               |
+=========================================+
Pilihan:
```

Gambar 4.4
Jika gagal 3 kali

Gambar 4.5

```
+================= LOGIN ====================+
| Masukkan username dan password untuk login |
+============================================+
Username: Bakil
Password: 044
Login berhasil, Selamat datang, Bakil (Admin)

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan:
```

Gambar 4.5
Jika berhasil login sebagai admin

Gambar 4.6

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 1

=== TIERLIST VALORANT ===
+----+----------------+------+-------------+----------+
| No | Nama Agent     | Tier | Role        | Pickrate |
+----+----------------+------+-------------+----------+
| 1  | Reyna          | A    | Duelist     | 60%      |
| 2  | Jett           | S    | Duelist     | 85%      |
| 3  | Iso            | B    | Duelist     | 40%      |
+----+----------------+------+-------------+----------+
```

Gambar 4.6

jika kita memilih opsi 1(read) pada menu admin

Gambar 4.7

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                           |
| 2. Tambah Agent                             |
| 3. Ubah Agent                               |
| 4. Hapus Agent                              |
| 5. Cari Agent                               |
| 6. Keluar                                   |
+=============================================+
Pilihan: 2
Nama Agent: Phoenix
Tier (S/A/B/C/D): S
Role: Duelist
Pickrate (%): 90
Agent berhasil ditambahkan.
```

Gambar 4.7
Jika kita memilih opsi 2 (create) pada menu admin

Gambar 4.8

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                           |
| 2. Tambah Agent                             |
| 3. Ubah Agent                               |
| 4. Hapus Agent                              |
| 5. Cari Agent                               |
| 6. Keluar                                   |
+=============================================+
Pilihan: 3
Nomor agent yang diubah: 3
Nama baru: Waylay
Tier baru: A
Role baru: Duelist
Pickrate baru: 85
Update berhasil.
```

Gambar 4.8
Jika kita memilih opsi 3 (Update) pada menu admin

Gambar 4.9

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 4
Nomor agent yang dihapus: 1
Agent dihapus.
```

Gambar 4.9
Jika kita memilih opsi 4 (delete) pada admin

Gambar 4.10

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 5

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 1
Masukkan nama agent: Jett

=== DATA AGENT DITEMUKAN ===
Nama     : Jett
Tier     : S
Role     : Duelist
Pickrate : 85%
```

Gambar 4.10
Jika kita memilih Cari agent berdasarkan nama dan ditemukan

Gambar 4.11

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 5

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 1
Masukkan nama agent: Reyna
Agent Reyna tidak ditemukan.
```

Gambar 4.11
Jika kita memilih Cari agent berdasarkan nama dan data tidak ditemukan

Gambar 4.11

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 5

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 1
Masukkan nama agent: Reyna
Agent Reyna tidak ditemukan.
```

Gambar 4.11
Jika kita memilih Cari agent berdasarkan nama dan data tidak ditemukan

Gambar 4.12

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 5

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 2
Masukkan tier (S/A/B/C/D): S

=== DATA AGENT DITEMUKAN ===
Nama     : Jett
Tier     : S
Role     : Duelist
Pickrate : 85%

=== DATA AGENT DITEMUKAN ===
Nama     : Phoenix
Tier     : S
Role     : Duelist
Pickrate : 90%
```

Gambar 4.12
Jika kita memilih Cari agent berdasarkan tier dan data ketemu

Gambar 4.13

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 5

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 2
Masukkan tier (S/A/B/C/D): C
Tidak ada agent dengan tier C.
```

Gambar 4.13
Jika kita memilih Cari agent berdasarkan tier dan data tidak ditemukan

Gambar 4.14

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Cari Agent                              |
| 6. Keluar                                  |
+============================================+
Pilihan: 6
Keluar dari menu Admin.

+========================================+
|   Selamat datang di Valorant Tierlist  |
+========================================+
| 1. Masuk                               |
| 2. Keluar                              |
+========================================+
Pilihan:
```

Gambar 4.14
Jika memilih opsi 6 (keluar) akan looping ke menu awal

Gambar 4.15

```
+================ MENU USER ================+
| 1. Lihat Tierlist                          |
| 2. Cari Agent                              |
| 3. Keluar                                  |
+============================================+
Pilihan: 1

=== TIERLIST VALORANT ===
+----+-----------------+------+-------------+----------+
| No | Nama Agent      | Tier | Role        | Pickrate |
+----+-----------------+------+-------------+----------+
| 1  | Jett            | S    | Duelist     | 85%      |
| 2  | Waylay          | A    | Duelist     | 85%      |
| 3  | Phoenix         | S    | Duelist     | 90%      |
+----+-----------------+------+-------------+----------+

+================ MENU USER ================+
| 1. Lihat Tierlist                          |
| 2. Cari Agent                              |
| 3. Keluar                                  |
+============================================+
Pilihan:
```

Gambar 4.15
Jika memilih read(lihat tierlist) pada menu user

Gambar 4.16

```
+================ MENU USER ================+
| 1. Lihat Tierlist                         |
| 2. Cari Agent                             |
| 3. Keluar                                 |
+===========================================+
Pilihan: 2

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 1
Masukkan nama agent: Waylay

=== DATA AGENT DITEMUKAN ===
Nama     : Waylay
Tier     : A
Role     : Duelist
Pickrate : 85%

+================ MENU USER ================+
| 1. Lihat Tierlist                         |
| 2. Cari Agent                             |
| 3. Keluar                                 |
+===========================================+
Pilihan:
```

Gambar 4.16
Jika kita memilih mencari agent berdasarkan nama dan berhasil pada menu user

Gambar 4.17

```
+================ MENU USER ================+
| 1. Lihat Tierlist                         |
| 2. Cari Agent                             |
| 3. Keluar                                 |
+===========================================+
Pilihan: 2

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 1
Masukkan nama agent: Iso
Agent Iso tidak ditemukan.
```

Gambar 4.17
Jika kita memilih mencari agent berdasarkan nama dan data tidak ditemukan di menu user

Gambar 4.18

```
+================ MENU USER ================+
| 1. Lihat Tierlist                         |
| 2. Cari Agent                             |
| 3. Keluar                                 |
+===========================================+
Pilihan: 2

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 2
Masukkan tier (S/A/B/C/D): S

=== DATA AGENT DITEMUKAN ===
Nama     : Jett
Tier     : S
Role     : Duelist
Pickrate : 85%

=== DATA AGENT DITEMUKAN ===
Nama     : Phoenix
Tier     : S
Role     : Duelist
Pickrate : 90%
```

Gambar 4.18
Jika kita memilih mencari agent berdasarkan tier dan berhasil pada menu user

Gambar 4.19

```
+================ MENU USER ================+
| 1. Lihat Tierlist                         |
| 2. Cari Agent                             |
| 3. Keluar                                 |
+===========================================+
Pilihan: 2

+---------------------------+
|        CARI AGENT         |
+---------------------------+
| 1. Cari berdasarkan Nama  |
| 2. Cari berdasarkan Tier  |
+---------------------------+
Pilihan: 2
Masukkan tier (S/A/B/C/D): D
Tidak ada agent dengan tier D.
```

Gambar 4.19
Jika kita memilih mencari agent berdasarkan tier dan data tidak ditemukan di menu user

Gambar 4.20

```
+-----------------------------+
|         SORTING AGENT       |
+-----------------------------+
| 1. Sort Tier (S -> D)       |
| 2. Sort Pickrate (kecil->bsr)|
| 3. Sort Nama (A -> Z)       |
+-----------------------------+
Pilihan: 2
Data berhasil diurutkan berdasarkan Pickrate (rendah -> tinggi).

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: 1

=== TIERLIST VALORANT ===
+----+----------------+------+-------------+----------+
| No | Nama Agent     | Tier | Role        | Pickrate |
+----+----------------+------+-------------+----------+
| 1  | Iso            | B    | Duelist     | 40%      |
| 2  | Kay-O          | A    | Initiator   | 45%      |
| 3  | Reyna          | A    | Duelist     | 60%      |
| 4  | Sage           | B    | Sentinel    | 67%      |
| 5  | Jett           | S    | Duelist     | 85%      |
+----+----------------+------+-------------+----------+
```

Gambar 4.20
Jika sorting agent berdasarkan pickrate lalu lihat tierlist barunya (kecil ke besar)

Gambar 4.21

```
+-----------------------------+
|         SORTING AGENT       |
+-----------------------------+
| 1. Sort Tier (S -> D)       |
| 2. Sort Pickrate (kecil->bsr)|
| 3. Sort Nama (A -> Z)       |
+-----------------------------+
Pilihan: 1
Data berhasil diurutkan berdasarkan Tier (S -> D).

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: 1

=== TIERLIST VALORANT ===
+----+----------------+------+-------------+----------+
| No | Nama Agent     | Tier | Role        | Pickrate |
+----+----------------+------+-------------+----------+
| 1  | Jett           | S    | Duelist     | 85%      |
| 2  | Kay-O          | A    | Initiator   | 45%      |
| 3  | Reyna          | A    | Duelist     | 60%      |
| 4  | Sage           | B    | Sentinel    | 67%      |
| 5  | Iso            | B    | Duelist     | 40%      |
+----+----------------+------+-------------+----------+
```

Gambar 4.21
Jika sorting agent berdasarkan Tier dan read tierlist baru (S ke D)

Gambar 4.22

```
+-----------------------------+
|         SORTING AGENT       |
+-----------------------------+
| 1. Sort Tier (S -> D)       |
| 2. Sort Pickrate (kecil->bsr)|
| 3. Sort Nama (A -> Z)       |
+-----------------------------+
Pilihan: 3
Data berhasil diurutkan berdasarkan Nama (A -> Z).

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: 1

=== TIERLIST VALORANT ===
+----+-----------------+------+-------------+----------+
| No | Nama Agent      | Tier | Role        | Pickrate |
+----+-----------------+------+-------------+----------+
| 1  | Iso             | B    | Duelist     | 40%      |
| 2  | Jett            | S    | Duelist     | 85%      |
| 3  | Kay-O           | A    | Initiator   | 45%      |
| 4  | Reyna           | A    | Duelist     | 60%      |
| 5  | Sage            | B    | Sentinel    | 67%      |
+----+-----------------+------+-------------+----------+
```

Gambar 4.22

Jika sorting agent berdasarkan Nama dan read tierlist barunya (A sampai Z)

Gambar 4.23

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: 6

+-----------------------------------+
|          SEARCHING AGENT          |
+-----------------------------------+
| 1. Binary Search (cari Pickrate)  |
| 2. Linear Search (cari Role)      |
+-----------------------------------+
Pilihan: 1
Masukkan pickrate yang dicari (%): 67

=== HASIL BINARY SEARCH (Pickrate: 67%) ===
Nama     : Sage
Tier     : B
Role     : Sentinel
Pickrate : 67%
---
```

Gambar 4.23

Jika Searching agent dengan binarySearch (mencari berdasarkan pickrate)

Gambar 4.24

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+==========================================+
Pilihan: 6

+-----------------------------------+
|          SEARCHING AGENT          |
+-----------------------------------+
| 1. Binary Search (cari Pickrate)  |
| 2. Linear Search (cari Role)      |
+-----------------------------------+
Pilihan: 2
Masukkan role yang dicari: Duelist

=== HASIL LINEAR SEARCH (Role: Duelist) ===
Nama     : Iso
Tier     : B
Role     : Duelist
Pickrate : 40%
---
Nama     : Reyna
Tier     : A
Role     : Duelist
Pickrate : 60%
---
Nama     : Jett
Tier     : S
Role     : Duelist
Pickrate : 85%
---
```

Gambar 4.24
Jika Searching agent dengan linearSearch (mencari berdasarkan Role)

Gambar 4.25

```
+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: vesvgse
>> Kesalahan: Input harus berupa angka.

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+============================================+
Pilihan: 2
Nama Agent: 0
Tier (S/A/B/C/D): 0
>> Kesalahan: Tier "0" tidak dikenali! Tier yang tersedia hanya S, A, B, C, atau D.
```

Gambar 4.25
error handling jika input tidak berupa angka pada menu admin dan error handling pada input angka pada pemilihan tier

Gambar 4.26

```
+=========================================+
Pilihan: 2
Nama Agent: 0
Tier (S/A/B/C/D): S
Role: 2
Pickrate (%): o
>> Kesalahan: Input pickrate harus berupa angka.

+================ MENU ADMIN ================+
| 1. Lihat Tierlist                          |
| 2. Tambah Agent                            |
| 3. Ubah Agent                              |
| 4. Hapus Agent                             |
| 5. Sorting Agent                           |
| 6. Searching Agent                         |
| 7. Cari Agent                              |
| 8. Keluar                                  |
+=========================================+
Pilihan: 0
>> Kesalahan: Pilihan tidak valid.
```

Gambar 4.26
error handling jika input pickrate tidak berupa angka dan error handling jika input menu admin tidak ada di pilihan

5. Langkah-langkah GIT

5.1 GIT Init

```
PS D:\Praktikum-APL> git init
Reinitialized existing Git repository in D:/Praktikum-APL/.git/
```

Gambar 5.1
Inisialisasi menggunakan git init

5.2 GIT Add

```
PS D:\Praktikum-APL> git add .
```

Gambar 5.2

Tambahkan kode dan lain lain ke repository setelah inisialisasi

5.3 GIT Commit

```
PS D:\Praktikum-APL> git commit -m "ALL LAST COMMIT"
[main 6c90412] ALL LAST COMMIT
 4 files changed, 902 insertions(+), 70 deletions(-)
 create mode 100644 POST-TEST/POSTTEST-7-APL/2509106044-MBakilAmru-PT-7.cpp
 create mode 100644 POST-TEST/POSTTEST-7-APL/2509106044-MBakilAmru-PT-7.exe
 create mode 100644 POST-TEST/POSTTEST-7-APL/errorhandling.h
```

Gambar 5.3
selanjutnya kita commit ke repository kita dengan catatannya

5.4 GIT Remote

```
PS D:\Praktikum-APL> git remote add origin https://github.com/BakilTechh/Praktikum-APL.git
```

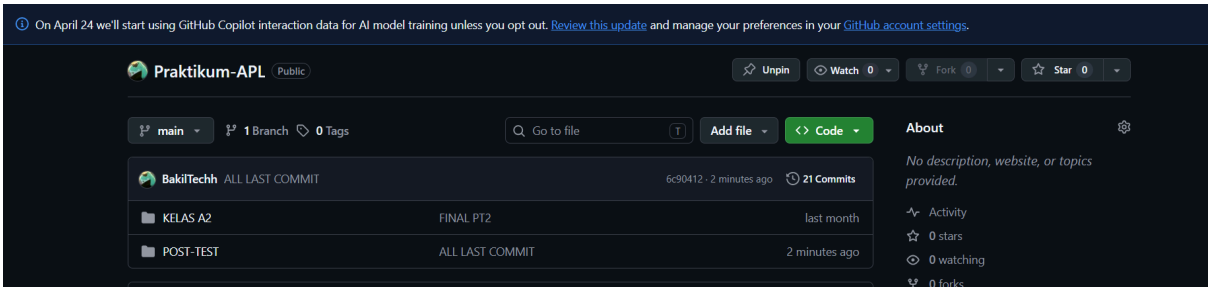
Gambar 5.4
Remote untuk mengarahkan yang kita mau tambahkan ke repo (hanya pertama kali)

5.5 GIT Push

```
PS D:\Praktikum-APL> git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 62.24 KiB | 3.66 MiB/s, done.
Total 9 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/BakilTechh/Praktikum-APL.git
   1da171e..6c90412  main -> main
```

Gambar 5.5
Terakhir jika semua sudah lansung kita push origin main semuanya

5.6 Tampilan GITHUB



Gambar 5.6
Begini hasil akhir dan Tampilan pada githubnya jika sudah